CSE 232, Homework 2

Name:

Student ID:

1. (30pts) State whether each of the following statements are true or false. Additionally, briefly explain your reasoning.

   (a) In causal broadcast, if a process p broadcasts a message m, then p will never need to buffer m. In other words, if p delivers m immediately after broadcasting m, it doesn't violate the properties of causal broadcast.

   [Answer: **False.** If the process has previously broadcast a message that it has not yet delivered, then it has to wait for that message to be delivered first.]

   (b) If there is no concurrency and no failures, then a regular register and an atomic register are equivalent.

   [Answer: **True.** Both provide the same guarantees under no concurrency/failures. It is only under concurrency/failures that atomic registers provide stronger guarantees than atomic registers.]

   (c) Consensus is possible in the fail-stop model even if a majority of processes fail.

   [Answer: **True.** For example, Consensus I and Consensus II are algorithms for consensus in a fail-stop model. They both implement consensus when even up to $(n-1)$ processes fail.]

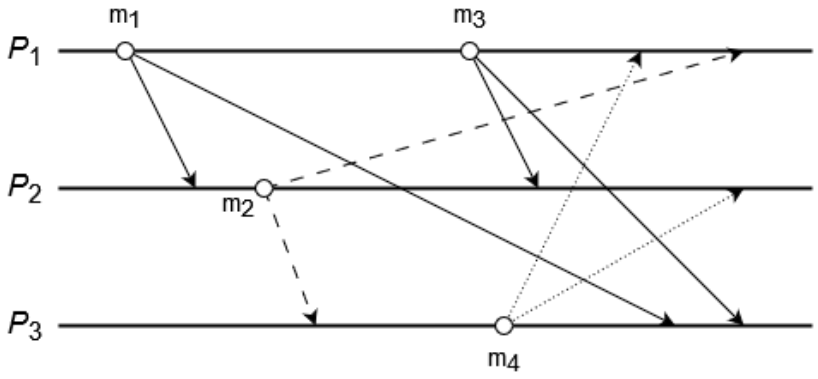
   (d) Uniform Reliable Broadcast in the fail-silent model is possible even if a majority of processes fail.

   [Answer: **False.** As we saw in the class. URB is requires at least a majority of correct processes within a fail-silent model.]

   (e) Wait-free algorithms are only possible in synchronous networks.

   [Answer: **False.** For example, we saw an example of a wait-free implementation of a regular register in asynchronous networks, though it required a correct majority.]

2. (24pts) Consider the execution below in the context of the causal broadcast protocol using vector clocks and answer the following questions. There is no distinction between solid, dashed, and dotted arrows; they are only for visual clarity. Assume messages are delivered as soon as possible, as long as the properties of causal broadcast are not violated. Assume each vector clock is initialized to all zeros.



(a) (4pts) For each message, give the vector clock attached to that message.
(*Notation*: $m_1(P_1, P_2)$ means message m_1 from process P_1 to process P_2)
[Answer: See the attached handwritten solution.]

$m_1(P_1, P_2)$: [Answer: $(0, 0, 0)$] $m_1(P_1, P_3)$: [Answer: $(0, 0, 0)$]

$m_2(P_2, P_1)$: [Answer: $(1, 0, 0)$] $m_2(P_2, P_3)$: [Answer: $(1, 0, 0)$]

$m_3(P_1, P_2)$: [Answer: $(1, 0, 0)$] $m_3(P_1, P_3)$: [Answer: $(1, 0, 0)$]

$m_4(P_3, P_1)$: [Answer: $(0, 0, 0)$] $m_4(P_3, P_2)$: [Answer: $(0, 0, 0)$]

(b) (12pts) For each pair of messages m and m', state whether (1) m is causally before m' (*i.e.*, $m \prec m'$), (2) m' is causally before m (*i.e.*, $m' \prec m$), or (3) m and m' are not causally related. If the messages are causally related, write the property (FIFO, InOut, transitivity) that relate them.

m_1, m_2 : [Answer: (1) $m_1 \prec m_2$, InOut] m_2, m_3 : [Answer: (3) not related]

m_1, m_3 : [Answer: (1) $m_1 \prec m_3$, FIFO] m_2, m_4 : [Answer: (3) not related]

m_1, m_4 : [Answer: (3) not related] m_3, m_4 : [Answer: (3) not related]

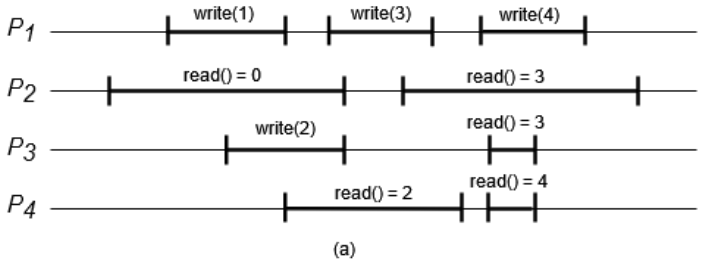
(c) (8pts) We wish to measure how often our messages need to buffer across the entire system. For each message m, count the number of times a different message m' is delivered between when m is received and m is delivered. List each message m' that is delivered in between. (For example, in an execution where a process P_6 delivers m_6 between receiving m_7 and delivering m_7, then the answer for "m_7 for P_6" would be $1, \{m_6\}$.)

m_1 for P_2: [Answer: 0] m_1 for P_3: [Answer: 0]

m_2 for P_1: [Answer: 0] m_2 for P_3: [Answer: $2, \{m_1, m_4\}$]

m_3 for P_2: [Answer: 0] m_3 for P_3: [Answer: 0]

3. (26pts) Consider the following executions of a register. Read and Write events are marked with the span of the operation as well as the return values for reads. Determine if each execution satisfies the properties of a regular register, and then if it satisfies the properties of an atomic register. If so, mark the linearization points. Explain your answer. Assume the initial value at each process is 0.



(a)

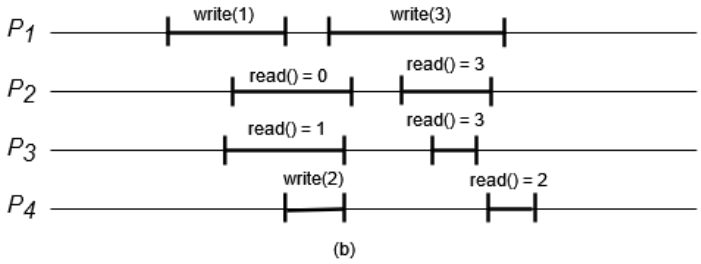
[Answer: **Both Atomic and regular.** Linearization points should be marked on the figure but here I am typing in latex. Consider the sequence of linearization points as a sequential execution. The following is their sequence:)

read() = 0 → write(1) → write(2) → read() = 2 →

write(3) → read() = 3 → read() = 3 →

write(4) → read() = 4

Every atomic register is a regular register.]



(b)

[Answer: **Not atomic, but regular.** They need to explain why it is not atomic:

- Consider the read() = 3 in P_3.
- For this to happen, the write(3) in P_1 must complete before then.
- Since the read() = 3 in P_3 finishes before the read() = 2 in P_4 begins, then that implies that the write(3) in P_1 also finishes before the read() = 2 in P_4.
- Thus, read() = 2 should return 3 as well which violates sequential specification.

Thus, this execution cannot be linearized, and cannot be atomic.

They need to explain why it is regular: It is easy to see that each read() either reads the latest written value or a concurrently written value.

- read() = 0 is reading the previous (initial) value of 0.
- read() = 1 at P_3 is reading the concurrent write(1).
- read() = 3 at P_2 and P_3 are reading the concurrent write(3).
- read() = 2 at P_4 is reading the previous write(2).

]

4. (20pts) Consider the consensus protocol for partially synchronous systems using the eventual perfect failure detector ◇P (Protocol III) that we saw in class. As a reminder, the leader executes the following in order to decide:

   (a) read the latest adopted value from a majority.

   (b) impose that value at a majority.

   (c) decide and broadcast the decision to all.

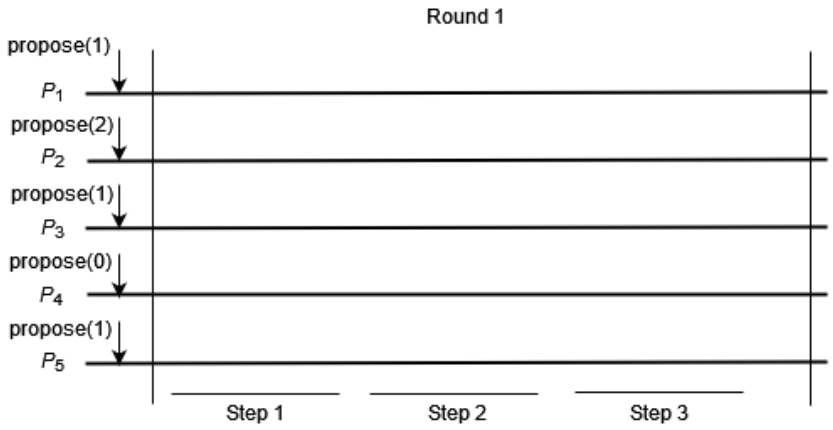
   Leaders are chosen is sequence: p_1 in round 1, p_2 in round 2 and so on. The following crashes happen during the execution:

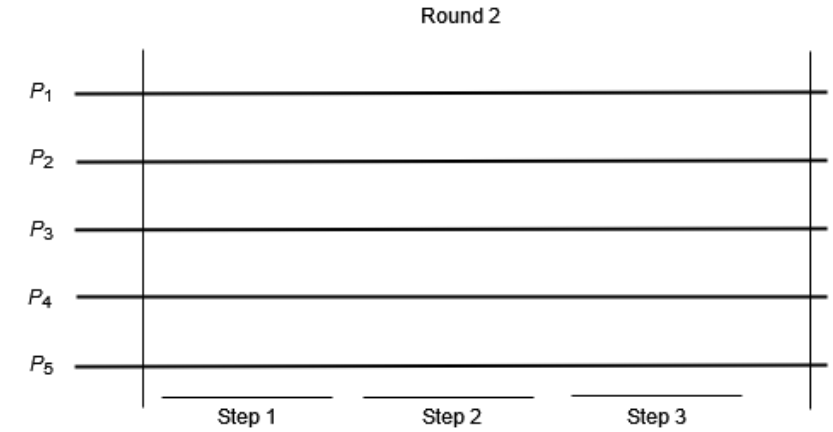
   - Round 1: Process p_1 crashes after step 1.
   - Round 2: Processes p_1 and p_3 crash at the beginning of the round, and restore at the end of the round. Process p_2 crashes after step 2.
   - Round 3: Processes p_4 and p_5 crash at the beginning of the round.
   - Round 4: No crashes.
   - Round 5: No crashes.

   Write in the diagram of each round

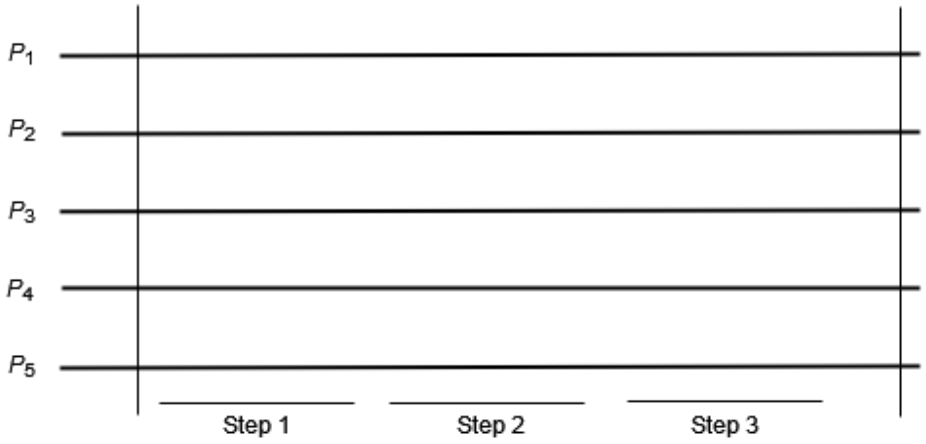
   - **the value of each process at the start of the round.**
   - **draw the messages and the attached value and time stamp (if any) that are sent in each step. For example, value 7 with timestamp 2 is written as $\langle 2, 7 \rangle$.**
   - **write the decide events if any process(es) decides during that round,**

   Do this up until round 5, or until every correct process decides – whichever happens first, i.e. you do not necessarily need to fill all 5 diagrams. [Answer: See the attached handwritten solution.]

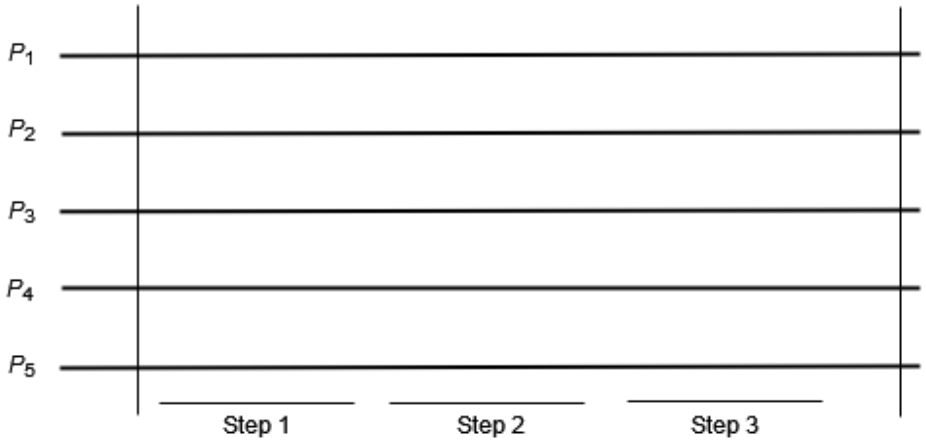




Round 3



Round 4



Round 5